

Module 5: Shapley Additive Explanations

Instructional Hours: 6

Module Overview

This module will provide you with a detailed review of Shapley Additive Explanations. You will learn how Shapley values are calculated and how they are used to explain the output of a AI model. You will also learn how you can implement Shapley additive explanations in python.

Learning Outcomes

At the end of this module, you will be able to:

1. Explain the principles of Shapley Additive Explanations (SHAP)
2. Describe the process of creating Shapley Additive Explanations
3. Create code to implement Shapley Additive Explanations using python
4. Interpret SHAP explanation outputs to stakeholders

Learning Activities/task List

1. Review Introductory notes on Shapley Additive Explanations
2. Contribute to a discussion on SHAP
3. Review selected readings on SHAP and Shapley Values
4. Complete a Quiz on SHAP
5. Implement Python code to generate Shapley values and their plots
6. Complete a Final Quiz on SHAP

Introduction To Shapley Additive Explanations

SHAP is both a local and Global model-agnostic explainability method that you use to interpret the output of a machine learning model. You use the method to explain how each feature contributes to a prediction. It is based on Shapley values, a concept from cooperative game theory. It measures the contribution of each player (feature) to the total payout (model prediction). It works by measuring the difference in prediction when a feature is included or excluded from the model.

In cooperative game theory, Shapley values are used to distribute a total payout among players based on their contribution to the game. In SHAP, the "game" is the machine learning model, the "players" are the features, and the "payout" is the model's prediction. In a machine learning model, the payout is equated to the gain in the prediction of an instance where the gain is the difference between the predicted value and the average value for all instances. Shapley values are calculated by considering all possible subsets of feature (values) and measuring how much a feature (value) contributes to the prediction.

Calculation of Shapley Values

The following are the sequence of steps used to calculate Shapley values:

1. Baseline Prediction

Start by considering the model's baseline prediction. This might be a constant prediction like the mean of all predictions. The goal is to compare how each feature changes this baseline.

2. Form All Feature Subsets (Coalitions)

For each feature i , you need to evaluate its contribution by considering all possible subsets of the other features. Let's say you have n features; for each feature i , you calculate how adding that feature to various subsets of the other features changes the model's prediction.

As an example, assume you have three features (A, B and C) with values $A = a_i$, $B = b_i$ and $C = c_i$. Features set values without a_i are $\{, \{b_i\}, \{c_i\}, \{bc_i\}$.

3. Marginal Contribution of a Feature

For a given subset of features values S (that does not include feature value a_i from the example in step ii), calculate the model's prediction with and without feature value a_i .

The marginal contribution of feature value a_i is the difference between the model's prediction when the feature value a_i is included in S and when it's not. This is represented by the following formula:

$$\text{Marginal Contribution}(a_i, S) = f(S \cup a_i) - f(S)$$

Where $f(S \cup a_i)$ is the models prediction when a_i is included and $f(S)$ is the model's prediction when a_i is excluded.

4. Shapley Value Calculation

Step iii calculates the marginal contribution of a feature value for each possible subset of feature values. All these have to be combined to obtain one Shapley value for all the combinations of features sets. This is obtained using the formula below. Note that each marginal contribution is weighted for the purpose of calculating an average value.

Where m is the number of possible subsets without feature A (including the empty subset), S is the set of all subsets without a_i . $|S_k|$ is the size of a subset and N is the number of features. $\frac{|S_k|!(N-|S_k|-1)!}{N!}$ is used to weigh the marginal value of each subset such that the total of all weights add up to 1.

This process is repeated for each of the feature values to get the Shapley value of each of them i.e. the Shapley value of b_1 and c_1 in our example.

Note that in the above example, there is an empty subset $\{\}$. The model prediction for this subset $f(\{\})$ is the base value of the average prediction for all the values in the dataset.

Once the Shapley values have been calculated, they are used to generate various kinds of explanation diagrams such as summary, dependence and force plots. You should then be able to interpret the information depicted in these diagrams.

Computational Challenges

Shapley values require computing all possible subsets, which grows exponentially as the number of features increases. In addition, it requires training a model for all possible combination of features. For n features, there are 2^n possible subsets.

To make this computation feasible in practice, approximation methods such as Monte Carlo sampling are often used. This approximates the Shapley values by sampling a subset of feature combinations rather than computing all of them.

Shapley Values Properties

Efficiency

The efficiency property of Shapley value means that the Shapley values of each of the features values must add up to the difference between the predicted value and the average value. For our example case

$$f(a_1, b_1, c_1) - E_x(f(x)) = SV(a_1) + SV(b_1) + SV(c_1)$$

Where $E_x(f(x))$ is the average of the prediction of all instances in the dataset.

Symmetry

This requires that for any two feature values j and k , their contributions should be the same if they contribute equally to all possible coalitions.

Dummy

A feature j that does not change the predicted value for any of the coalition of feature values it is added to should have a Shapley value of 0

Additivity

If a model consists of a number of models that are combined (Ensemble), then the Shapley value for the entire model can be computed and averaging the Shapley values for each of the sub-models in the ensemble.

SHAP Plots

SHAP Feature Importance

Feature importance is directly proportional to the features Shapley value. A feature value with a larger Shapley value is considered more important. To get the global importance, you average the Shapley values per feature for all instances in the dataset (mean absolute Shapley value). These values for all the features can be sorted and plotted for instance using a horizontal bar chart. The most important features will be at the top of the bar chart.

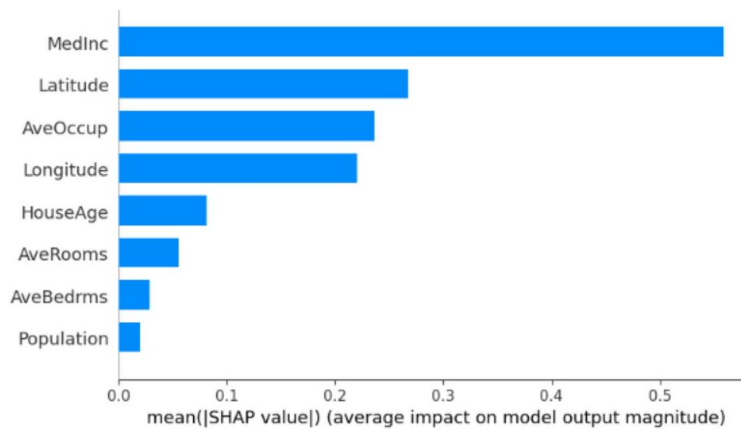


Figure 5.1: Showing that the median income of flat residents is the most importance feature in predicting the price of a house in a flat.

SHAP Summary Plot

This plot summarizes the SHAP values for all features across the entire dataset. It shows which features are the most important in the model and the direction of their influence (positive or negative impact on the prediction). Color coding (e.g., red for high feature values, blue for low) helps interpret how different feature values impact predictions.

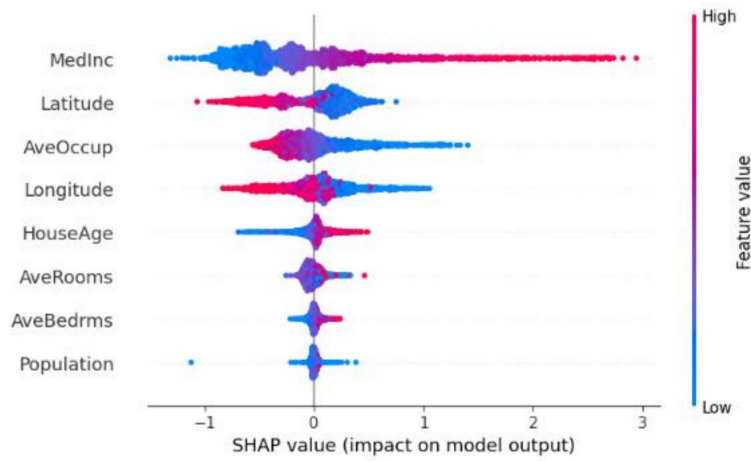


Figure 5.2: Summary Plot of SHAP values for the California Housing Data Set.

From Figure 5.2, you can see that for the MedInc attribute, a low value results in a low Shapley value. For the Latitude, it is the opposite. A high value results in a low Shapley value.

SHAP Dependence Plot

The dependence plot shows how SHAP values change with respect to one feature and how that feature dependence on another feature in the dataset. It is a plot of the feature value on the x -axis and the Shapley value on the y axis. Color coding is used to show the dependence on another feature of interest. It helps answer the question - how does the change (from low to high) of the feature value impact the prediction?

And how are the Shapley values of the feature of interest impacted by the changes on another feature of interest.

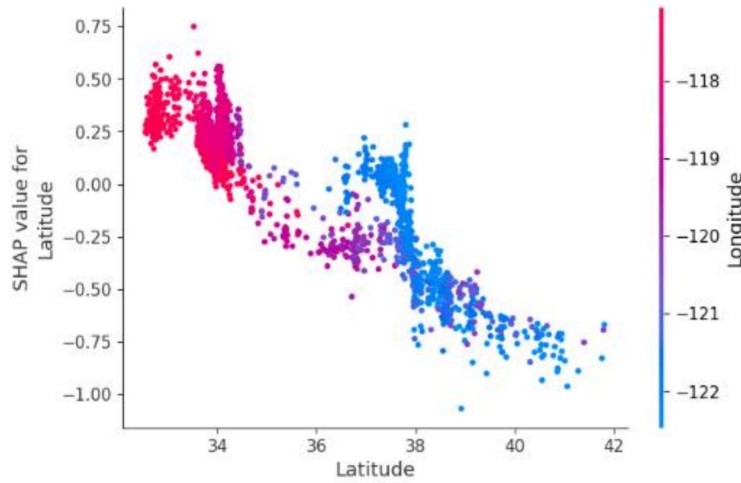


Figure 5.3: Dependence Plot of SHAP values for Latitude feature in the California Housing Data Set showing interaction with the longitude feature.

From Figure 5.3, you can see that a low latitude and a high longitude results in a higher price contribution for the latitude feature.

SHAP Force Plot

This plot visualizes how individual features contribute to a single prediction. It displays the baseline prediction and how much each feature pushes the prediction higher or lower, offering a clear picture of feature contribution for that instance.



Figure 5.4: A force plot for an instance in California Housing Data Set. MedInc exerts the most force towards reducing the price.

Table 5.1: Shapley Values corresponding to the instance in Figure 5.4

MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
-0.830	0.003	-0.012	-0.008	-0.002	-0.244	-0.323	-119

Detailed Review of SHAP and Shapley Values

Reading Material

To learn about SHAP and Shapley Values, read the following materials.

Shapley Values

Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.), URL:
<https://christophm.github.io/interpretable-ml-book/shapley.html>

SHAP (Shapley Additive exPlanations)

Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.), URL:
<https://christophm.github.io/interpretable-ml-book/shap.html>

Implementation of SHAP Using Python

You can implement the SHAP explanation technique using python. You should make use of the shap package. In the code below, you will use the shap package to calculate Sharpley values for each instance in the data set. You will the use the package to create a number of plots. You will rely on the California housing dataset from the sklearn datasets.

```
#import the necessary packages
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor as RFR
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap
from sklearn.metrics import accuracy_score

#get the data set
data = fetch_california_housing()

print(data.DESCR)
#print(data.feature_names)
```

```

#print(data.target_names)

#Check the dimensions of the data
x=data.data
print(x.shape)

y=data.target
#Put the data in a dataframe for easier visualization
import pandas as pd
columns=data.feature_names.copy()
df = pd.DataFrame(x, columns=columns)
df['target'] = y

df.head(10)

#get a summary of the data
df.describe()

#Print the correlation matrix
corrM=df.corr()
sns.heatmap(corrM, annot=True, cmap='coolwarm', fmt='.2f')
plt.show()

# split the data into training and test set
x=df[columns]
y=df['target']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.1, random_state=42)

# fit the model
model=RFR(n_estimators=100, random_state=42)
model = model.fit(x_train, y_train)

#print(model.summary())
importances=[]
for col, val in zip(columns,model.feature_importances_):
    importances.append((col, val))
importances.sort(key=lambda x: x[1], reverse=True)
print(importances)

```



```

#perform some predictions
predictions=model.predict(x_test)

print("R Squared={:.2f}".format(model.score(x_test, y_test)))

# Initialize SHAP explainer
explainer = shap.TreeExplainer(model)

# Calculate SHAP values using the test data
shap_values = explainer(x_test)
#print the Sharpley value for the first instance in the test data
print(columns)
print(shap_values.values[0])

#print the average/base value
print("Base Value={:.3f}".format(shap_values.base_values[0]))

#predict the price of the first instance in the test date

print("Predicted Price= {:.3f}".format(model.predict([x_test.iloc[0]])[0]))
print("Actual={:.2f}".format(y_test.iloc[0]))

#Plot the feature importances
shap.summary_plot(shap_values, x_test, plot_type='bar')

#Create the summary plot
shap.summary_plot(shap_values, x_test)

# Generate the dependence plot of Latitude on longitude
shap.dependence_plot("Latitude", shap_values.values, x_test, interaction_index="Longitude")

#Generate the force plot for the first instance in the test data
shap.initjs()
shap.force_plot(explainer.expected_value,shap_values.values[0,:], x_test.iloc[0,:])

```

